KUBERNETES

Kubernetes (K8s) is an open-source platform that automates the deployment, scaling, and management of containerized applications. Originally developed by Google, it orchestrates containers across clusters of virtual or physical machines, providing self-healing, load balancing, and automated rollouts.

Key features and concepts of Kubernetes include:

- **Container Orchestration:** It manages the lifecycle of containerized applications, grouping them into logical units (pods) for easy management and discovery.
- **Automation & Scaling:** Kubernetes automatically scales applications up or down based on demand and automates application updates and rollbacks
- **Self-Healing:** If a container fails, Kubernetes restarts it; if a node dies, it replaces and reschedules containers.
- **Portability:** It runs on-premises, in public clouds (e.g., AWS, Azure, GCP), or hybrid environments, allowing consistent application deployment.
- **Cluster Structure:** It organizes nodes (machines) into a cluster managed by a control plane, ensuring efficient resource utilization.

Kubernetes is essential for managing microservices at scale and is widely adopted as the standard for cloud-native applications

What is Docker vs Kubernetes? - Kubernetes is an orchestration system for running containers. It automates the process of deploying, operating and scaling containerized applications. Docker is a platform for creating containerized applications. Applications can be run manually using the Docker runtime or via an orchestrator.

Container orchestrators such as Kubernetes help manage the complexity of the container lifecycle. This becomes especially important when you're operating distributed applications with large numbers of containers across multi-cloud environments.
Kubernetes automates deployment, load balancing, resource allocation, observability and security enforcement for containers via declarative configuration and automation. It keeps containerized applications running in the desired state, ensuring they are scalable and resilient.

For more info on Docker vs Kubernetes – see this

Kubernetes is a portable, extensible, open source platform for managing containerized workloads and services that facilitate both declarative configuration and automation. It has a large, rapidly growing ecosystem. Kubernetes services, support, and tools are widely available.

Why you need Kubernetes and what it can do:

Containers are a good way to bundle and run your applications. In a production environment, you need to manage the containers that run the applications and ensure that there is no downtime. For example, if a container goes down, another container needs to start. Wouldn't it be easier if this behavior was handled by a system?

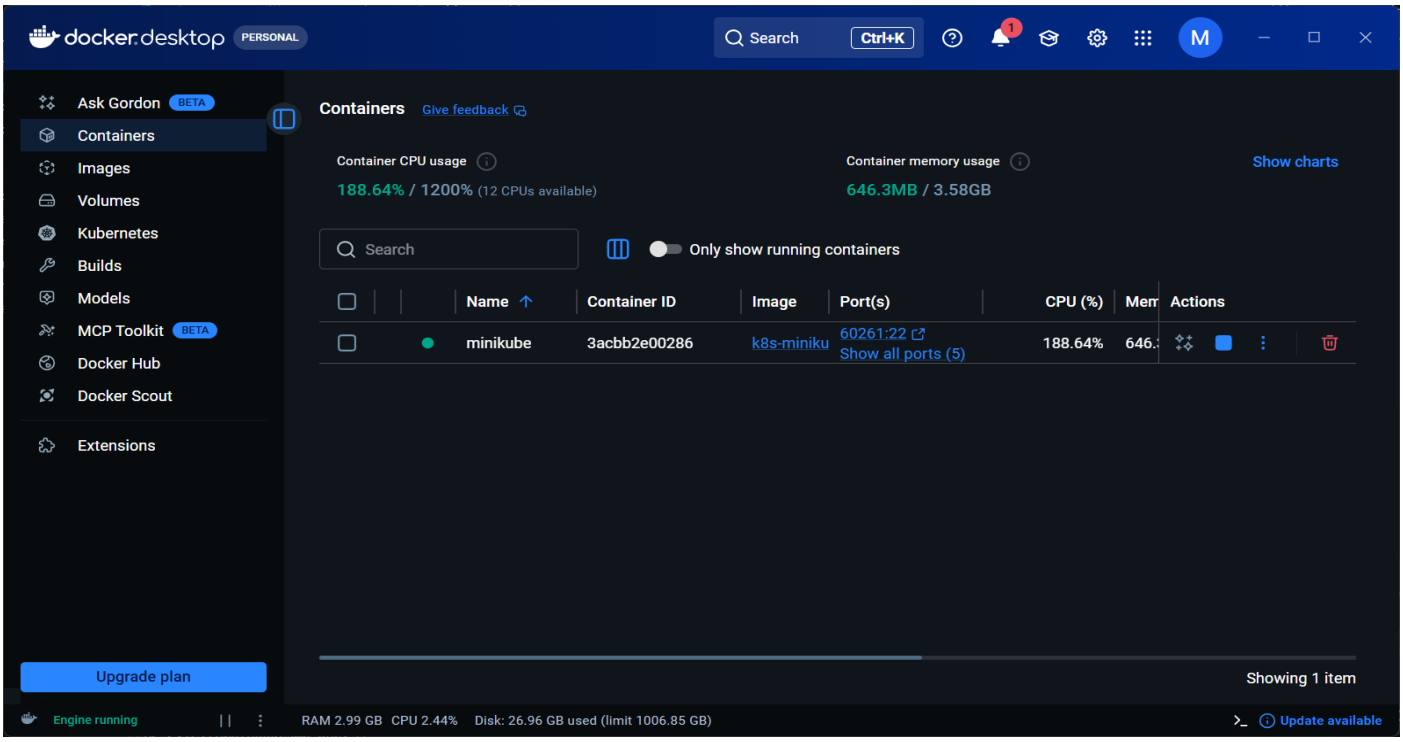
That's how Kubernetes comes to the rescue! Kubernetes provides you with a framework to run distributed systems resiliently. It takes care of scaling and failover for your application, provides deployment patterns, and more. For example: Kubernetes can easily manage a canary deployment for your system.

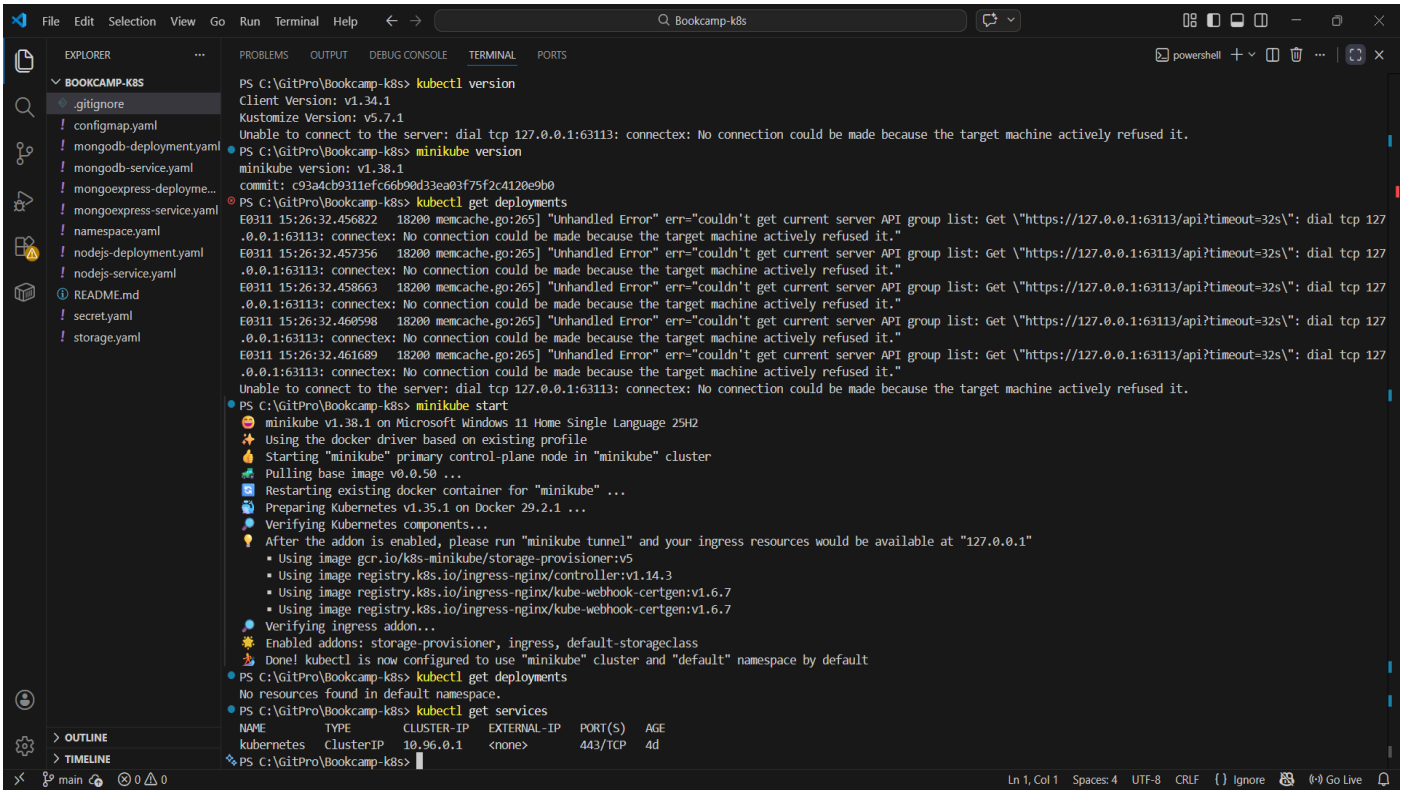
For more information on Kubernetes-see this overview

Bookcamp-K8s:

Step1: Install minikube and kubectl in windows.

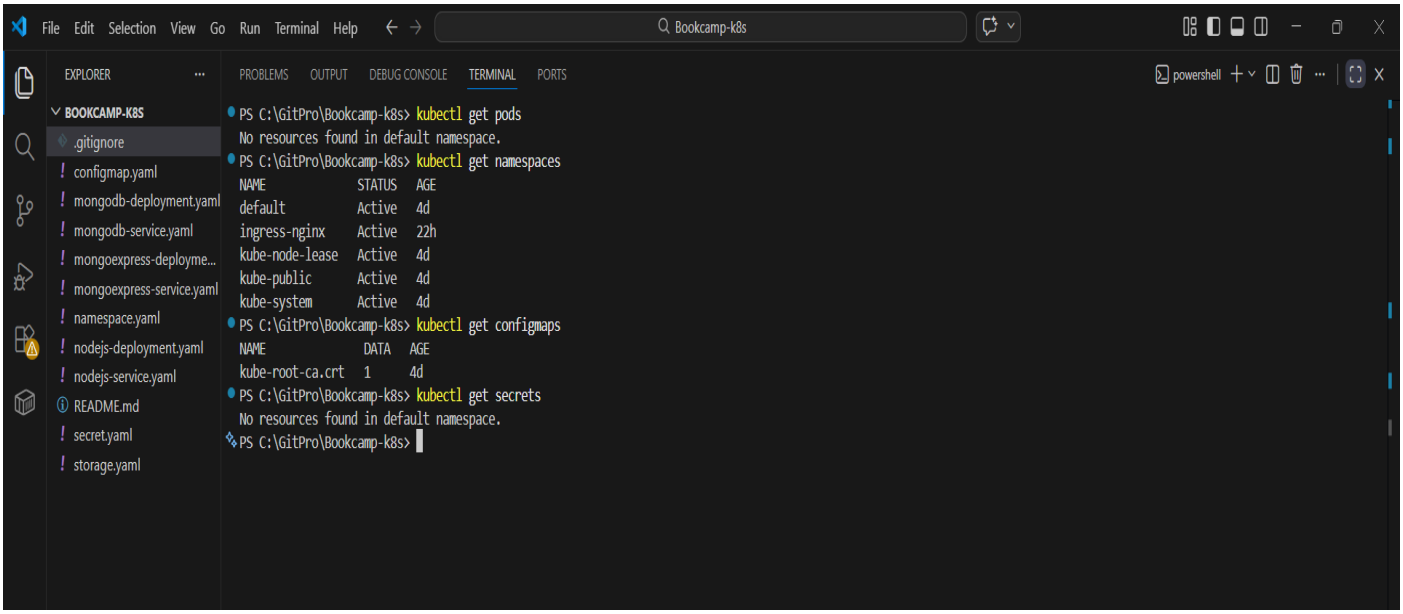
- ✓ Download .exe file from here and install minikube then add it to path (environment variables).
- ✓ Close and restart terminal and check minikube using - minikube version.
- ✓ Open powershell and run this - curl.exe -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/windows/amd64/kubectl.exe"
- ✓ move the downloaded kubectl.exe file into C:\kubectl folder. then add it to path in environment variables.
- ✓ Verify using - kubectl version.
- ✓ Now run the docker desktop app and open terminal and run command – minikube start or minikube start --driver=docker.





Basic Kubectl Commands:

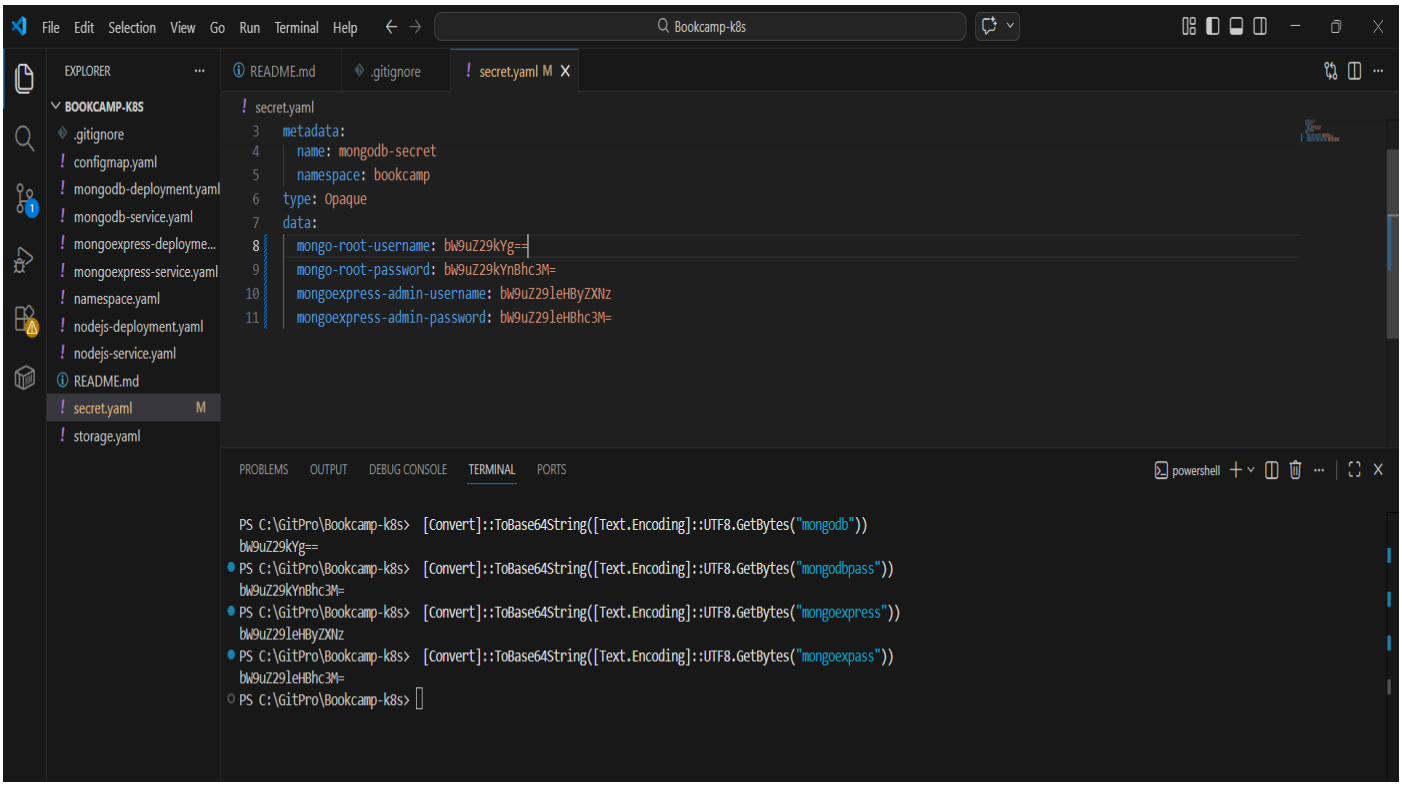
Kubectl get pods #lists running containers/pods
Kubectl get deployments #lists running deployments
Kubectl get services/svc #lists all services
Kubectl get persistentvolumeclaims/pvc #lists all persistent volumes
Kubectl get secrets #lists all secrets
Kubectl get configmaps #lists all configmaps
Note: to list these components in a specific namespace use "-n namespace" in all commands
Kubectl get namespaces #lists all namespaces



Step2: now get the base64 encoded credentials for mongodb connection and mongoexpress access/login

- Use this command to convert text to base64 encode
  [convert]::ToBase64String([Text.Encoding]::UTTF8.GetBytes("your text"))

- Now place these base64 encode texts or usernames into secret file and change config map MONGO_URL with your mongodb credentials which are base64 coded just now in a plain text



Step3: Now run the pods and deployment and services use commands
Kubectl apply -f deployment-filename.yaml #creates deployment and runs it
Kubectl apply -f service-filename.yaml #creates service for the pods/deployment to communicate
Kubectl apply -f namespace.yaml #creates a namespace and all these deployments run in this namespace

```
PS C:\GitPro\Bookcamp-k8s> kubectl apply -f namespace.yaml
namespace/bookcamp created
PS C:\GitPro\Bookcamp-k8s> kubectl get namespaces
NAME              STATUS   AGE
bookcamp          Active   7s
default           Active   4d1h
ingress-nginx     Active   22h
kube-node-lease   Active   4d1h
kube-public       Active   4d1h
kube-system       Active   4d1h
PS C:\GitPro\Bookcamp-k8s> kubectl apply -f secret.yaml
secret/mongodb-secret created
PS C:\GitPro\Bookcamp-k8s> kubectl get secrets -n bookcamp
NAME             TYPE     DATA   AGE
mongodb-secret   Opaque   4      7s
PS C:\GitPro\Bookcamp-k8s> kubectl apply -f configmap.yaml
configmap/mongodb-config created
PS C:\GitPro\Bookcamp-k8s> kubectl get configmaps -n bookcamp
NAME               DATA   AGE
kube-root-ca.crt   1      35s
mongodb-config     1      6s
PS C:\GitPro\Bookcamp-k8s> kubectl apply -f storage.yaml
persistentvolumeclaim/mongodb-pvc created
PS C:\GitPro\Bookcamp-k8s> kubectl get persistentvolumeclaims -n bookcamp
NAME          STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
mongodb-pvc   Bound    pvc-1f9f628f-ff7b-47cb-9b87-b91213355e2b   1Gi        RWO            standard       <unset>                 11s
PS C:\GitPro\Bookcamp-k8s> kubectl get pvc -n bookcamp
NAME          STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
mongodb-pvc   Bound    pvc-1f9f628f-ff7b-47cb-9b87-b91213355e2b   1Gi        RWO            standard       <unset>                 42s
PS C:\GitPro\Bookcamp-k8s>
```

Verify if components are created using - kubectl get command (see in images)

```
PS C:\GitPro\Bookcamp-k8s> kubectl apply -f mongodb-deployment.yaml
deployment.apps/mongodb created
PS C:\GitPro\Bookcamp-k8s> kubectl get deployments -n bookcamp
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
mongodb   1/1     1            1           17s
PS C:\GitPro\Bookcamp-k8s> kubectl get pods -n bookcamp
NAME                       READY   STATUS    RESTARTS   AGE
mongodb-5fd94d8588-bg42f   1/1     Running   0          34s
PS C:\GitPro\Bookcamp-k8s> kubectl apply -f mongodb-service.yaml
service/mongodb created
PS C:\GitPro\Bookcamp-k8s> kubectl get services -n bookcamp
NAME      TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)     AGE
mongodb   ClusterIP   10.103.227.202   <none>        27017/TCP   18s
PS C:\GitPro\Bookcamp-k8s> kubectl get svc -n bookcamp
NAME      TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)     AGE
mongodb   ClusterIP   10.103.227.202   <none>        27017/TCP   28s
PS C:\GitPro\Bookcamp-k8s> kubectl apply -f nodejs-deployment.yaml
deployment.apps/nodejs created
PS C:\GitPro\Bookcamp-k8s> kubectl get deployments -n bookcamp
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
mongodb   1/1     1            1           2m32s
nodejs    2/2     2            2           13s
PS C:\GitPro\Bookcamp-k8s> kubectl get pods -n bookcamp
NAME                       READY   STATUS    RESTARTS   AGE
mongodb-5fd94d8588-bg42f   1/1     Running   0          2m36s
nodejs-76fc54d57c-7v9nv    1/1     Running   0          17s
nodejs-76fc54d57c-lckbr    1/1     Running   0          17s
PS C:\GitPro\Bookcamp-k8s> kubectl apply -f nodejs-service.yaml
service/nodejs-service created
PS C:\GitPro\Bookcamp-k8s> kubectl get services -n bookcamp
NAME             TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
mongodb          ClusterIP   10.103.227.202   <none>        27017/TCP        119s
nodejs-service   NodePort    10.108.248.38    <none>        5000:30007/TCP   6s
PS C:\GitPro\Bookcamp-k8s> kubectl get svc -n bookcamp
NAME             TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
mongodb          ClusterIP   10.103.227.202   <none>        27017/TCP        2m9s
nodejs-service   NodePort    10.108.248.38    <none>        5000:30007/TCP   16s
PS C:\GitPro\Bookcamp-k8s>
```

```
PS C:\GitPro\Bookcamp-k8s> kubectl apply -f mongoexpress-deployment.yaml
deployment.apps/mongo-express created
PS C:\GitPro\Bookcamp-k8s> kubectl get deployments -n bookcamp
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
mongo-express   1/1     1            1           27s
mongodb         1/1     1            1           5m7s
nodejs          2/2     2            2           2m48s
PS C:\GitPro\Bookcamp-k8s> kubectl get pods -n bookcamp
NAME                            READY   STATUS    RESTARTS   AGE
mongo-express-fbb9f454b-6kfr5   1/1     Running   0          40s
mongodb-5fd94d8588-bg42f        1/1     Running   0          5m20s
nodejs-76fc54d57c-7v9mv         1/1     Running   0          3m1s
nodejs-76fc54d57c-lckbr         1/1     Running   0          3m1s
PS C:\GitPro\Bookcamp-k8s> kubectl apply -f mongoexpress service.yaml
error: Unexpected args: [service.yaml]
See 'kubectl apply -h' for help and examples
PS C:\GitPro\Bookcamp-k8s> kubectl apply -f mongoexpress-service.yaml
service/mongo-express-service created
PS C:\GitPro\Bookcamp-k8s> kubectl get services -n bookcamp
NAME                  TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
mongo-express-service NodePort    10.97.185.49    <none>        8081:30008/TCP   20s
mongodb               ClusterIP   10.103.227.202  <none>        27017/TCP        5m51s
nodejs-service        NodePort    10.108.248.38   <none>        5000:30007/TCP   3m58s
PS C:\GitPro\Bookcamp-k8s>
```

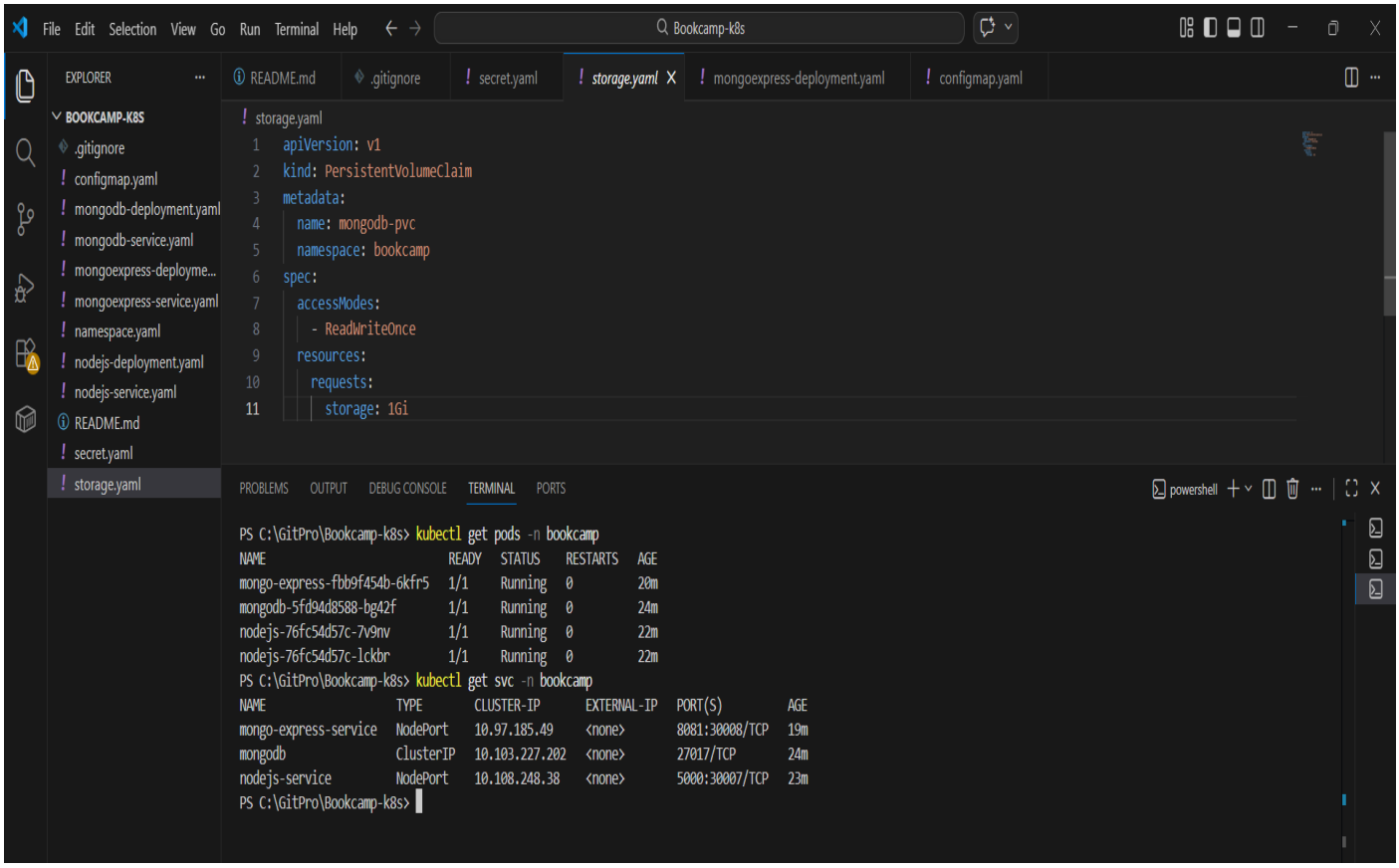
Start the service using – minikube service servicename -n namespace
Get service-names using – kubectl get svc -n namespace
minikube will directly creates an tunnel and opens application in browser by default.
To know the ip address of minikube use – minikube ip or minikube service servicename --url -n namespace

```
PS C:\GitPro\Bookcamp-k8s> kubectl get svc -n bookcamp
NAME                  TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
mongo-express-service NodePort    10.97.185.49    <none>        8081:30008/TCP   78s
mongodb               ClusterIP   10.103.227.202  <none>        27017/TCP        6m49s
nodejs-service        NodePort    10.108.248.38   <none>        5000:30007/TCP   4m56s
PS C:\GitPro\Bookcamp-k8s> minikube service nodejs-service -n bookcamp
|  NAMESPACE  |  NAME  |  TARGET PORT  |  URL  |
|  bookcamp  |  nodejs-service  |  5000  |  http://192.168.49.2:30007  |
Starting tunnel for service nodejs-service.
|  NAMESPACE  |  NAME  |  TARGET PORT  |  URL  |
|  bookcamp  |  nodejs-service  |  |  http://127.0.0.1:54514  |
Opening service bookcamp/nodejs-service in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

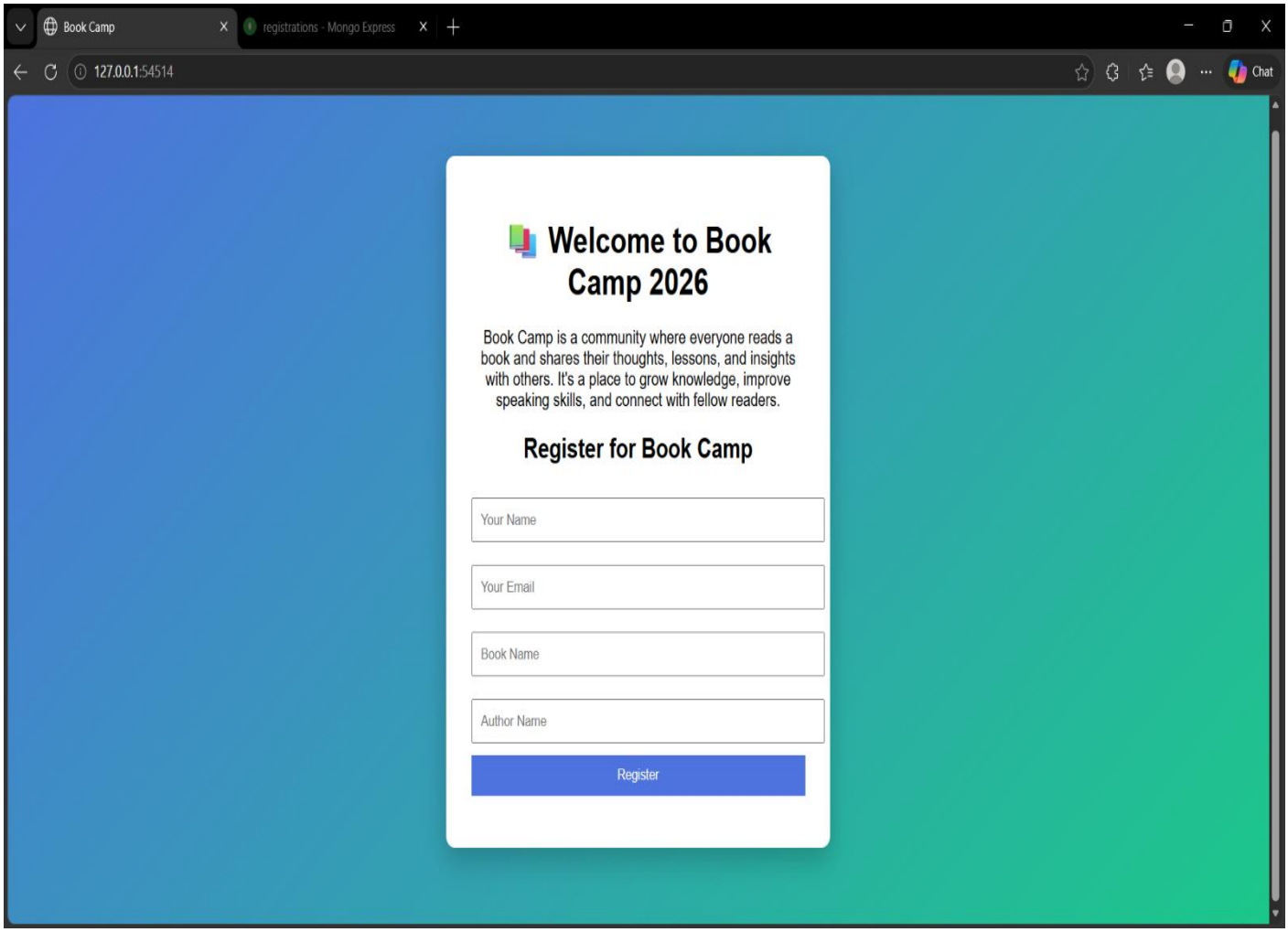
Run the application first using minikube command and open new terminal and run mongoexpress UI minikube service.
If you want run it in background using – minikube service svcname -n namespace &
Now browse the address - http://<minikubeip>:tunnel
Ex: http://192.68.49.2:30008 for application and http://192.168.49.2:30007
Or browse http://127.0.0.1:tunnel | http://localhost:tunnel

```
PS C:\GitPro\Bookcamp-k8s> kubectl get svc -n bookcamp
NAME                  TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
mongo-express-service NodePort    10.97.185.49    <none>        8081:30008/TCP   2m18s
mongodb               ClusterIP   10.103.227.202  <none>        27017/TCP        7m49s
nodejs-service        NodePort    10.108.248.38   <none>        5000:30007/TCP   5m56s
PS C:\GitPro\Bookcamp-k8s> minikube service mongo-express-service -n bookcamp
|  NAMESPACE  |  NAME  |  TARGET PORT  |  URL  |
|  bookcamp  |  mongo-express-service  |  8081  |  http://192.168.49.2:30008  |
Starting tunnel for service mongo-express-service.
|  NAMESPACE  |  NAME  |  TARGET PORT  |  URL  |
|  bookcamp  |  mongo-express-service  |  |  http://127.0.0.1:60423  |
Opening service bookcamp/mongo-express-service in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
```

Verify all running pods using - kubectl get pods -n namespace
Verify all running services using - kubectl get svc -n namepace



Browse the application and mongoexpress and use mongoexpress credentials to authenticate



Book Camp | 127.0.0.1:60423

127.0.0.1:60423

Sign in to access this site

Authorization required by http://127.0.0.1:60423

Username: mongoexpress

Password: ••••••••••

Sign in | Cancel

Book Camp | Home - Mongo Express

127.0.0.1:60423

Mongo Express Database

Mongo Express

Databases

Database Name | + Create Database

View	admin	Del
View	bookcamp	Del
View	config	Del
View	local	Del

Server Status

Hostname	mongodb-5fd94d8588-bg42f	**MongoDB Version**	7.0.30
Uptime	693 seconds	**Node Version**	18.20.3
Server Time	Wed, 11 Mar 2026 10:45:49 GMT	**V8 Version**	10.2.154.26-node.37
Current Connections	9	**Available Connections**	48003

Book Camp | registrations - Mongo Express

127.0.0.1:60423/db/bookcamp/registrations

Mongo Express Database: bookcamp > Collection: registrations

Viewing Collection: registrations

New Document | New Index

Simple | Advanced

Key | Value | String | Find

No documents found.

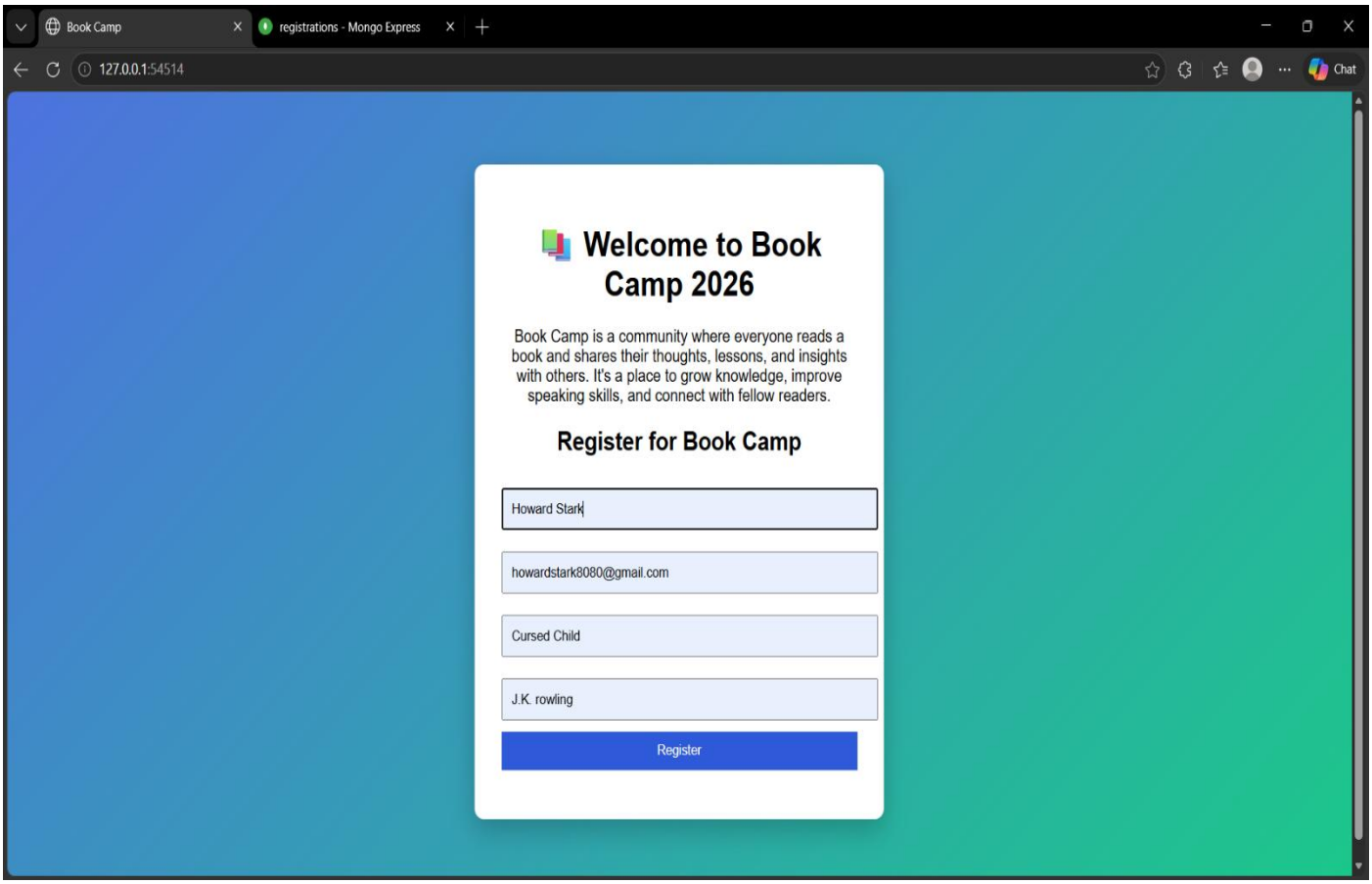
Rename Collection

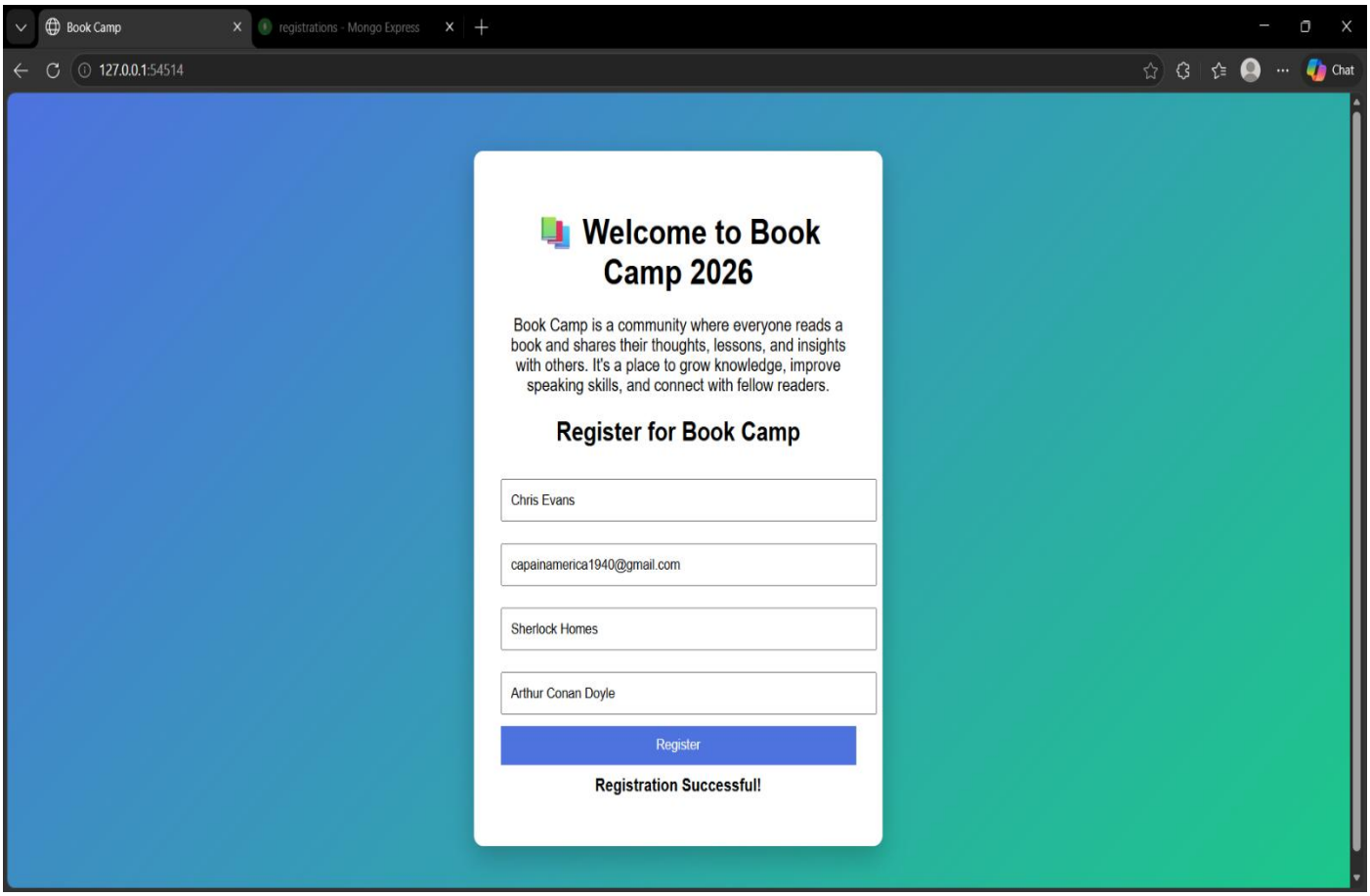
bookcamp . registrations | Rename

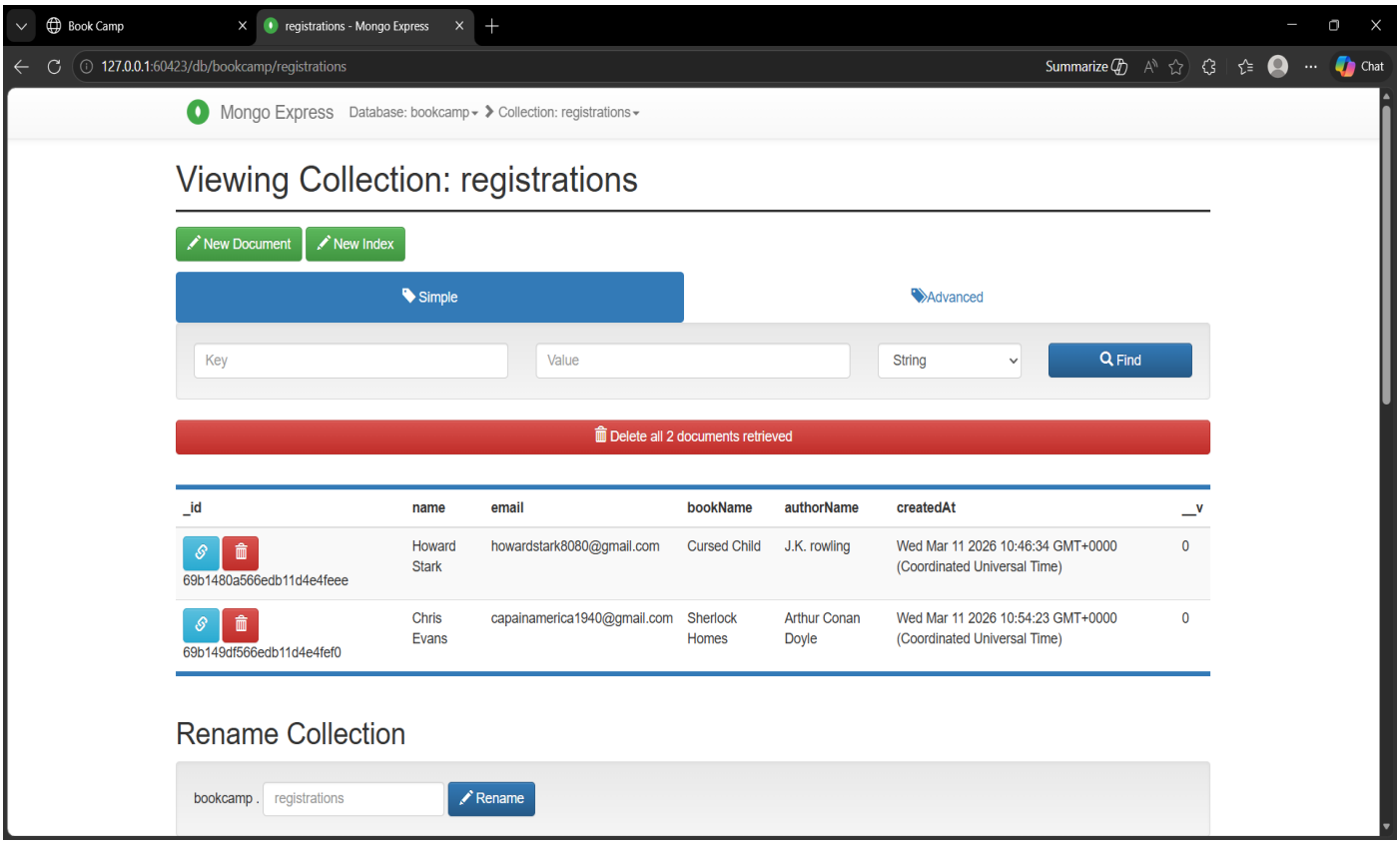
Tools

Export Standard | Export --jsonArray | Export --csv

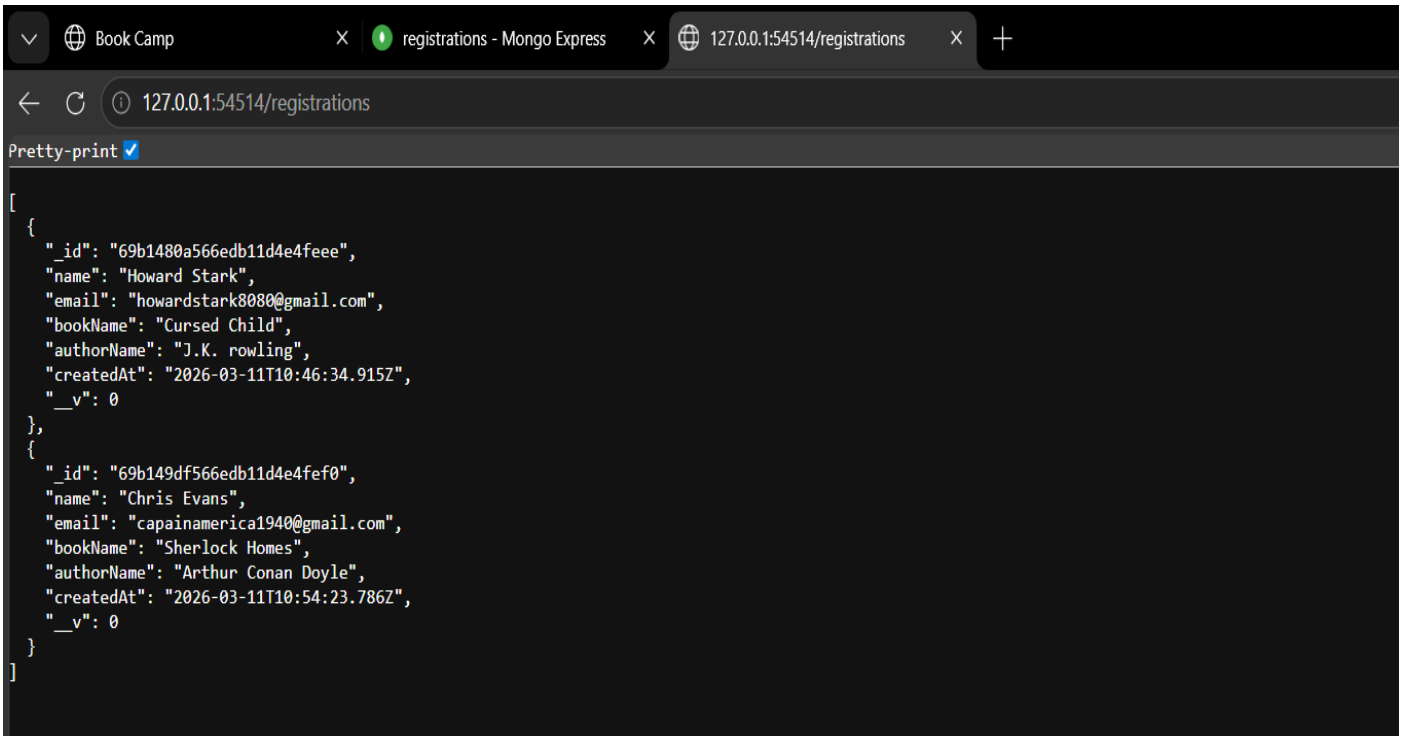
Now try to register enter some details and check in mongoexpress if data successfully stored







Or optionally browse <application-url>/registrations for json format data



To close the application use ctrl+c in terminal
To delete component use – kubectl delete component component-name -n namespace
To delete all application use – kubectl delete namespace namespace-name
Ex: Kubectl delete namespace bookspace #deletes all components created in this bookcamp namespace.